

PANDUAN LENGKAP

Web App dari Nol sampai Production

React Vite · Prisma · PostgreSQL · Docker · Dokploy · GitHub CI/CD

Panduan profesional mencakup seluruh alur: Planning & PRD, UI/UX Design, Frontend, Backend, Database, Auth, Dockerisasi, CI/CD, VPS, Deployment, hingga Maintenance. Dilengkapi perintah terminal lengkap, checklist, dan praktik terbaik production.

Layer	Teknologi	Keterangan
Frontend	React + Vite	SPA, build static, served via Nginx
Backend API	Express / Fastify	REST API, port 3001, TypeScript
Database	PostgreSQL 16	Container terpisah, port 5432
ORM	Prisma	Schema as code, type-safe query
Cache	Redis 7	Session, rate-limit, queue (optional)
Reverse Proxy	Traefik	SSL otomatis via Let's Encrypt
Orchestration	Dokploy	UI management di atas Docker
Registry	GitHub GHCR	Docker image store
CI/CD	GitHub Actions	Build → push image → deploy otomatis

Daftar Isi

1. Planning & Arsitektur

- PRD.md
- Sitemap & User Flow
- Database Schema
- Tech Architecture
- API Contract
- Env Structure

2. UI/UX Design

- Wireframe
- Design System & Tokens
- Component Mapping
- Responsive Breakpoints

3. Setup Project & Struktur Folder

- Monorepo structure
- Struktur Frontend
- Struktur Backend

4. Frontend — React Vite

- Init & dependencies
- Vite config + proxy
- Axios interceptor
- Error Boundary
- Nginx production config

5. Backend — API, Prisma & Auth

- Init server
- Prisma schema & migrate
- Express entry point
- Auth middleware
- Global error handler

6. Dockerisasi

- Dockerfile Frontend
- Dockerfile Backend
- docker-compose.yml
- dockerignore
- Build & test lokal

7. Local Testing & Validasi

- Health check
- Checklist sebelum push
- Security check

8. Git & CI/CD

- Branching strategy
- gitignore
- GitHub Actions workflow
- GitHub Secrets

9. VPS & Infrastruktur

- Setup & hardening VPS
- Install Docker
- Install Dokploy
- Domain & SSL
- Backup otomatis

10. Deploy Production

- Checklist pre-deploy
- Alur staging → production
- Verifikasi setelah deploy

11. Maintenance & Terminal

- Perintah terminal wajib
- Cek database
- Update aplikasi
- DB optimization
- Scaling strategy
- Checklist rutin

FASE 1

Planning & Arsitektur

1.1 PRD.md — Product Requirements Document

Buat file PRD.md di root project sebelum menulis satu baris kode pun. Ini adalah sumber kebenaran tunggal seluruh tim.

```
■ docs/PRD.md

# Product Requirements Document

## Overview
Nama Produk : [nama]
Target User : [siapa]
Problem      : [masalah yang diselesaikan]
Solution     : [solusi kita]

## Goals & Success Metrics
- Metric 1: 1.000 user aktif bulan pertama
- Metric 2: Response API < 200ms

## Features MVP
| No | Feature   | Priority | Status |
|----|-----|-----|-----|
| 1  | Auth     | P0      | TODO  |
| 2  | Dashboard| P0      | TODO  |

## Out of Scope (v1)
- Fitur X ditunda ke v2

## Timeline
Week 1-2 : Planning + Design
Week 3-5 : Development
Week 6   : Testing + Deploy
```

1.2 Database Schema Plan

```
■ docs/schema-plan.md

## Entities
User      : id, name, email, passwordHash, role, createdAt
Project   : id, title, userId (FK), status, createdAt
Task      : id, title, projectId (FK), assigneeId, dueDate

## Relations
User 1 --< Project (one to many)
Project 1 --< Task (one to many)

## Indexes
- users.email UNIQUE
- tasks.projectId (performance)
```

1.3 Tech Architecture

```
■ Alur request production
```

Browser

```
■■ HTTPS ■■■ Traefik (SSL termination)
    ■■ /      ■■■ Frontend (Nginx, port 80)
    ■■ /api/* ■■■ Backend  (Express, port 3001)
                    ■■ Prisma ORM
                        ■■ PostgreSQL (port 5432)
                        ■■ Redis (port 6379)
```

1.4 API Contract

■ docs/api-contract.md

```
POST /api/auth/register    body: { name, email, password }
POST /api/auth/login       body: { email, password }
GET  /api/auth/me          header: Authorization: Bearer <token>

GET  /api/projects
POST /api/projects         body: { title, description }
PUT  /api/projects/:id
DEL  /api/projects/:id
```

```
## Response format (selalu konsisten)
{ "success": true,  "data": {...} }
{ "success": false, "error": "pesan", "code": "ERR_CODE" }
```

1.5 Environment Structure

■ .env.example (commit ini. JANGAN commit .env asli)

```
NODE_ENV=development
PORT=3001
FRONTEND_URL=http://localhost:5173

DATABASE_URL=postgresql://user:pass@localhost:5432/dbname

JWT_SECRET=ganti_dengan_random_min_32_karakter
JWT_EXPIRES_IN=7d

REDIS_URL=redis://localhost:6379
ALLOWED_ORIGINS=http://localhost:5173,https://domain.com
```

FASE 2

UI/UX Design

■ Fase ini paling sering di-skip dan paling sering jadi penyebab frontend berantakan saat scale. Luangkan 1-2 hari di sini.

2.1 Wireframe

- Buat wireframe low-fidelity untuk setiap halaman di PRD
- Fokus ke layout dan hierarki konten — bukan warna
- Tools gratis: Figma (free tier), Excalidraw, Penpot
- Wajib ada versi mobile dan desktop untuk setiap halaman
- Simpan link Figma di docs/design.md

2.2 Design System & Tokens

■ src/styles/tokens.css

```
:root {
  /* Colors */
  --color-primary: #4F46E5;
  --color-success: #059669;
  --color-warning: #D97706;
  --color-error: #DC2626;
  --color-bg: #FFFFFF;
  --color-surface: #F8F9FC;
  --color-text: #1A1A2E;
  --color-muted: #6B7280;

  /* Typography */
  --font-sans: 'Inter', sans-serif;

  /* Spacing — 8px grid */
  --sp-1:4px; --sp-2:8px; --sp-3:12px;
  --sp-4:16px; --sp-6:24px; --sp-8:32px;

  /* Radius */
  --r-sm:6px; --r-md:10px; --r-lg:16px;
}
```

2.3 Component Mapping

Halaman	Komponen yang Diperlukan
Layout	Navbar, Sidebar, Footer, PageWrapper, Container
Auth	LoginForm, RegisterForm, InputField, Button, Alert
Dashboard	StatCard, ChartWidget, DataTable, Pagination
Profile	AvatarUpload, EditForm, PasswordChange
Common	Modal, Toast, Spinner, EmptyState, ErrorBoundary

2.4 Responsive Breakpoints

■ [tailwind.config.js](#) — screens

```
screens: {  
  'sm': '640px',    // Mobile landscape  
  'md': '768px',    // Tablet  
  'lg': '1024px',   // Desktop  
  'xl': '1280px',   // Large desktop  
}  
// Aturan: Mobile-first!  
// Tulis base style untuk mobile, override dengan md: lg:
```

FASE 3

Setup Project & Struktur Folder

■ Root monorepo

```
my-app/
  ■■■ frontend/          # React Vite app
  ■■■ backend/           # Express API server
  ■■■ docs/              # PRD, wireframe, API contract
  ■■■ docker-compose.yml
  ■■■ docker-compose.prod.yml
  ■■■ .env.example
  ■■■ .gitignore
  ■■■ README.md
```

■ frontend/ — detail

```
frontend/
  ■■■ public/
  ■■■ src/
    ■ ■■■ assets/        # Gambar, font, ikon
    ■ ■■■ components/
      ■ ■■■ common/      # Button, Input, Modal
      ■ ■■■ layout/       # Navbar, Sidebar, Footer
      ■ ■■■ features/     # Komponen per fitur
    ■ ■■■ hooks/         # Custom React hooks
    ■ ■■■ lib/           # axios instance, utils
    ■ ■■■ pages/         # Satu file per route
    ■ ■■■ store/         # Zustand / Redux
    ■ ■■■ styles/        # global.css, tokens.css
    ■ ■■■ types/         # TypeScript interfaces
    ■ ■■■ App.tsx
    ■ ■■■ main.tsx
  ■■■ .env
  ■■■ Dockerfile
  ■■■ nginx.conf
  ■■■ vite.config.ts
```

■ backend/ — detail

```
backend/
  ■■■ prisma/
    ■ ■■■ schema.prisma
    ■ ■■■ migrations/
  ■■■ src/
    ■ ■■■ controllers/   # Handler per resource
    ■ ■■■ middlewares/   # auth, error, rateLimit
    ■ ■■■ routes/
    ■ ■■■ services/      # Business logic
    ■ ■■■ lib/           # prisma client, redis, jwt
    ■ ■■■ types/
    ■ ■■■ utils/
    ■ ■■■ index.ts
  ■■■ .env
  ■■■ Dockerfile
  ■■■ package.json
```


FASE 4

Frontend — React + Vite

4.1 Init & Install Dependencies

■ Terminal

```
cd frontend
npm create vite@latest . -- --template react-ts
npm install

# Routing, state, HTTP client
npm install react-router-dom axios zustand

# Styling
npm install -D tailwindcss postcss autoprefixer @types/node
npx tailwindcss init -p
```

4.2 Vite Config — Alias & Proxy ke Backend

■ frontend/vite.config.ts

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import path from 'path'

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: { '@': path.resolve(__dirname, './src') },
  },
  server: {
    port: 5173,
    proxy: {
      '/api': { target: 'http://localhost:3001', changeOrigin: true },
    },
  },
})
```

4.3 Axios Instance + Token Interceptor

■ src/lib/api.ts

```
import axios from 'axios'

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || '/api',
  timeout: 10000,
})

// Inject JWT token ke setiap request
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// Handle 401 global – redirect ke login
```

```

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem('token')
      window.location.href = '/login'
    }
    return Promise.reject(err)
  }
)

export default api

```

4.4 Error Boundary Global

■ `src/components/common/ErrorHandler.tsx`

```

import { Component, ReactNode } from 'react'

export class ErrorBoundary extends Component<
  { children: ReactNode }, { hasError: boolean }
> {
  state = { hasError: false }

  static getDerivedStateFromError() {
    return { hasError: true }
  }

  render() {
    if (this.state.hasError) return (
      <div className='p-8 text-center'>
        <p>Terjadi kesalahan.</p>
        <button onClick={() => this.setState({ hasError: false })}>
          Coba Lagi
        </button>
      </div>
    )
    return this.props.children
  }
}

```

4.5 Nginx Config untuk Production

■ `frontend/nginx.conf`

```

server {
  listen 80;
  root /usr/share/nginx/html;
  index index.html;

  # Handle React Router (SPA fallback)
  location / {
    try_files $uri $uri/ /index.html;
  }

  # Cache aset statis 1 tahun
  location /assets/ {

```

```
    expires 1y;
    add_header Cache-Control 'public, immutable';
}

# Security headers
add_header X-Frame-Options      DENY;
add_header X-Content-Type-Options nosniff;
add_header X-XSS-Protection    '1; mode=block';
}
```

FASE 5

Backend — API, Prisma & Auth

5.1 Init Backend

■ Terminal

```
cd backend
npm init -y
npm install express cors helmet morgan bcryptjs jsonwebtoken dotenv
npm install -D typescript ts-node nodemon @types/express @types/node
npm install prisma @prisma/client
npx prisma init
```

5.2 Prisma Schema

■ backend/prisma/schema.prisma

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id          String   @id @default(cuid())
  name        String
  email       String   @unique
  passwordHash String
  role        Role      @default(USER)
  createdAt   DateTime @default(now())
  projects    Project[]
}

model Project {
  id        String   @id @default(cuid())
  title     String
  userId    String
  user      User      @relation(fields: [userId], references: [id])
  createdAt DateTime @default(now())
}

enum Role { USER ADMIN }
```

5.3 Perintah Prisma Lengkap

■ Terminal — Prisma commands

```
# Buat migration baru (development)
npx prisma migrate dev --name nama_migration

# Jalankan migration di production (tanpa interaksi)
npx prisma migrate deploy

# Generate Prisma Client setelah ubah schema
```

```
npx prisma generate

# Buka database GUI di browser
npx prisma studio

# Reset database (HATI-HATI: hapus semua data!)
npx prisma migrate reset
```

5.4 Express Server Entry Point

■ backend/src/index.ts

```
import express from 'express'
import cors from 'cors'
import helmet from 'helmet'
import morgan from 'morgan'
import { authRouter } from './routes/auth'
import { projectRouter } from './routes/projects'
import { errorHandler } from './middlewares/errorHandler'

const app = express()
const PORT = process.env.PORT || 3001

app.use(helmet())
app.use(cors({ origin: process.env.ALLOWED_ORIGINS?.split(',') }))
app.use(express.json())
app.use(morgan('combined'))

app.get('/api/health', (_, res) => res.json({ status: 'ok' }))
app.use('/api/auth', authRouter)
app.use('/api/projects', projectRouter)
app.use(errorHandler)

app.listen(PORT, () => console.log(`Server running on :${PORT}`))
```

5.5 Auth Middleware & Error Handler

■ backend/src/middlewares/auth.ts

```
import { Request, Response, NextFunction } from 'express'
import jwt from 'jsonwebtoken'

export const authenticate = (req: Request, res: Response, next: NextFunction) => {
  const token = req.headers.authorization?.split(' ')[1]
  if (!token) return res.status(401).json({ error: 'Token tidak ada' })
  try {
    (req as any).user = jwt.verify(token, process.env.JWT_SECRET!)
    next()
  } catch {
    res.status(401).json({ error: 'Token tidak valid' })
  }
}
```

■ backend/src/middlewares/errorHandler.ts

```
import { Request, Response, NextFunction } from 'express'

export const errorHandler = (err: any, req: Request, res: Response, _: NextFunction) => {
  console.error(err.stack)
```

```
res.status(err.status || 500).json({
  success: false,
  error: err.message || 'Internal Server Error',
  ...(process.env.NODE_ENV === 'development' && { stack: err.stack }),
})
}
```

FASE 6

Dockerisasi — Multi-service Compose

6.1 Dockerfile Frontend (multi-stage build)

■ frontend/Dockerfile

```
# Stage 1: Build React app
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Serve via Nginx (image ringan)
FROM nginx:alpine AS runner
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

6.2 Dockerfile Backend (multi-stage build)

■ backend/Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npx prisma generate
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/prisma ./prisma
COPY --from=builder /app/package.json .
EXPOSE 3001
# Jalankan migrate dulu, baru start server
CMD ["sh", "-c", "npx prisma migrate deploy && node dist/index.js"]
```

6.3 docker-compose.yml — Semua Service

■ docker-compose.yml

```
version: '3.9'

services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-user}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-pass}
```

```
    POSTGRES_DB:      ${POSTGRES_DB:-myapp}
volumes:
  - postgres_data:/var/lib/postgresql/data
healthcheck:
  test: ['CMD-SHELL', 'pg_isready -U user']
  interval: 10s
  retries: 5

redis:
  image: redis:7-alpine
  restart: unless-stopped

backend:
  build: ./backend
  restart: unless-stopped
  env_file: ./backend/.env
  ports: ['3001:3001']
  depends_on:
    db:
      condition: service_healthy

frontend:
  build: ./frontend
  restart: unless-stopped
  ports: ['80:80']
  depends_on: [backend]

volumes:
  postgres_data:
```

6.4 .dockerignore & Build Lokal

■ .dockerignore (frontend & backend)

```
node_modules
.git
.env
*.log
dist
.DS_Store
coverage
.github
```

■ Terminal — build dan jalankan lokal

```
docker compose build      # build semua image
docker compose up -d      # jalankan background
docker compose ps         # cek status
docker compose logs -f backend
docker compose down       # stop semua
```


FASE 7

Local Testing & Validasi

7.1 Health Check & Test API

■ Terminal — curl test

```
# Health check
curl http://localhost:3001/api/health
# Expected: {"status":"ok"}

# Register user baru
curl -X POST http://localhost:3001/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name":"Test","email":"test@test.com","password":"Pass1234!"}'

# Login
curl -X POST http://localhost:3001/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@test.com","password":"Pass1234!"}'
```

7.2 Checklist Sebelum Push ke GitHub

- Semua endpoint API berjalan di localhost
- Frontend terhubung ke backend — tidak ada CORS error
- Login, register, logout berfungsi
- Halaman tidak rusak di mobile (Chrome DevTools responsive mode)
- Error handler menampilkan pesan yang benar
- .env tidak akan ter-commit (ada di .gitignore)
- Tidak ada console.log berisi password atau token
- npm run build berhasil tanpa error (frontend)
- docker compose up berhasil dan semua service healthy

7.3 Security Basic Check

- Semua route sensitif menggunakan middleware authenticate
- Password di-hash dengan bcrypt (bukan disimpan plain text)
- JWT_SECRET minimal 32 karakter random
- Helmet.js aktif untuk menambah HTTP security headers
- Rate limiting aktif di endpoint /auth (prevent brute force)
- Prisma menggunakan parameterized query — aman dari SQL injection
- Input validation di setiap request body sebelum diproses

FASE 8

Git & CI/CD — GitHub Actions

Jangan deploy manual terus-menerus. Lama-lama bikin capek dan rawan error. Setup CI/CD dari awal.

8.1 Branching Strategy

■ Branch model

```
main      → Production (protected, JANGAN push langsung)
staging   → Staging environment
develop   → Integrasi semua feature
feature/* → Fitur baru (cabang dari develop)
fix/*     → Bug fix
hotfix/*  → Fix urgent di production (dari main)
```

Alur: feature/* → develop → staging → main

8.2 .gitignore

■ .gitignore (root)

```
.env .env.local .env.production
node_modules/
dist/ build/ .next/
.DS_Store Thumbs.db
*.log logs/
docker-compose.override.yml
```

8.3 GitHub Actions — Build, Push GHCR, Deploy

■ .github/workflows/deploy.yml

```
name: Build & Deploy
on:
  push:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMG_BE: ghcr.io/${{ github.repository }}/backend
  IMG_FE: ghcr.io/${{ github.repository }}/frontend

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4

      - name: Login GHCR
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
```

```
- name: Build & Push Backend
  uses: docker/build-push-action@v5
  with:
    context: ./backend
    push: true
    tags: ${ env.IMG_BE }:latest

- name: Build & Push Frontend
  uses: docker/build-push-action@v5
  with:
    context: ./frontend
    push: true
    tags: ${ env.IMG_FE }:latest
```

```
deploy:
  needs: build-and-push
  runs-on: ubuntu-latest
  steps:
    - name: Trigger Dokploy
      run: curl -X POST '${ secrets.DOKPLOY_WEBHOOK_URL }'
```

8.4 GitHub Repository Secrets

Secret Name	Keterangan
DOKPLOY_WEBHOOK_URL	URL webhook dari dashboard Dokploy → Settings
DATABASE_URL	PostgreSQL connection string untuk production
JWT_SECRET	Random string min. 32 karakter
POSTGRES_PASSWORD	Password database production

FASE 9

VPS & Infrastruktur

9.1 Setup & Hardening VPS

■ Terminal (setelah SSH ke VPS baru)

```
# Update sistem
sudo apt update && sudo apt upgrade -y

# Install tools dasar
sudo apt install -y curl git ufw fail2ban

# Setup Firewall
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow ssh
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 3000/tcp    # Dokploy dashboard
sudo ufw enable

# Nonaktifkan login password – gunakan SSH key saja
sudo nano /etc/ssh/sshd_config
# Set: PasswordAuthentication no
sudo systemctl restart ssh
```

9.2 Install Docker di VPS

■ Terminal

```
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker

# Verifikasi
docker --version
docker compose version
```

9.3 Install Dokploy

■ Terminal — satu perintah

```
curl -sSL https://dokploy.com/install.sh | sh

# Dokploy berjalan di port 3000
# Akses: http://IP-VPS:3000
# Buat akun admin pertama kali lewat browser
```

★ Setelah install, akses dashboard → Settings → Git Providers → hubungkan GitHub account.

9.4 Langkah di Dashboard Dokploy

#	Langkah
1	Create Project → beri nama project
2	Add Service → pilih Docker Compose atau Application

3	Hubungkan repository GitHub
4	Pilih branch: main (production) atau staging
5	Set semua Environment Variable (DATABASE_URL, JWT_SECRET, dll)
6	Set Build Type: Dockerfile atau Docker Compose
7	Save → Deploy pertama kali secara manual
8	Copy Webhook URL → masukkan ke GitHub Secrets

9.5 Domain & SSL Otomatis

- Di Cloudflare / Namecheap: arahkan A Record ke IP VPS
- Di Dokploy: Settings → Domains → tambah domain kamu
- Aktifkan HTTPS — Traefik + Let's Encrypt bekerja otomatis
- Tunggu 1-5 menit, SSL sudah aktif

■ Verifikasi domain

```
dig +short yourdomain.com
# Harus tampil IP VPS

curl -I https://yourdomain.com
# Harus: HTTP/2 200
```

9.6 Backup Database Otomatis

■ /home/user/backup.sh

```
#!/bin/bash
DATE=$(date +%Y%m%d_%H%M%S)
CONTAINER=myapp-db-1
BACKUP_DIR=/home/user/backups

mkdir -p $BACKUP_DIR
docker exec $CONTAINER pg_dump -U user myapp \
  | gzip > $BACKUP_DIR/db_${DATE}.sql.gz

# Hapus backup lebih dari 7 hari
find $BACKUP_DIR -name '*.sql.gz' -mtime +7 -delete
```

■ Cron job — backup setiap hari jam 02:00

```
chmod +x /home/user/backup.sh
crontab -e

# Tambahkan baris berikut:
0 2 * * * /home/user/backup.sh >> /home/user/backup.log 2>&1
```

FASE 10

Deploy Production

10.1 Checklist Sebelum Deploy

- Semua test lokal sudah lulus
- ENV production sudah diset di Dokploy (bukan nilai development)
- Tidak ada console.log yang bocorkan data sensitif
- Database backup terakhir sudah ada dan bisa di-restore
- docker compose build sukses tanpa error
- GitHub Actions CI berhasil (semua langkah hijau)
- Sudah dicoba dan berjalan di staging environment
- Health endpoint /api/health sudah ada dan berfungsi

10.2 Alur Deploy Staging → Production

■ Urutan yang benar

```
# 1. Push ke staging
git push origin develop:staging

# 2. Test di staging URL
curl https://staging.yourdomain.com/api/health

# 3. Jika OK, merge ke main
git checkout main
git merge develop
git push origin main

# 4. GitHub Actions otomatis:
#   Build Docker image
#   Push ke GHCR
#   Trigger Dokploy deploy

# 5. Monitor di Dokploy dashboard
#   Lihat logs deploy real-time
```

10.3 Verifikasi Setelah Deploy

■ Terminal — cek production

```
# Health check API
curl https://api.yourdomain.com/api/health

# Cek SSL valid
curl -I https://yourdomain.com

# SSH ke VPS, cek container
docker ps

# Lihat logs real-time
docker logs -f myapp-backend-1 --tail 50
```

FASE 11

Maintenance & Cek via Terminal

11.1 Perintah Terminal Wajib Hafal

Perintah	Fungsi
<code>docker ps</code>	Lihat semua container yang sedang berjalan
<code>docker ps -a</code>	Lihat semua container termasuk yang berhenti
<code>docker logs -f <nama></code>	Lihat log real-time sebuah container
<code>docker logs --tail 100 <nama></code>	100 baris log terakhir
<code>docker stats</code>	Monitor CPU & RAM tiap container secara live
<code>docker exec -it <nama> sh</code>	Masuk terminal di dalam container
<code>docker compose restart <svc></code>	Restart satu service tertentu
<code>docker compose pull && up -d</code>	Update image dan restart semua service
<code>docker system prune -f</code>	Hapus image/container tidak terpakai
<code>df -h</code>	Cek sisa disk space VPS
<code>free -h</code>	Cek penggunaan RAM VPS
<code>htop</code>	Monitor proses server secara interaktif
<code>netstat -tlnp</code>	Lihat port yang sedang aktif / listen
<code>curl localhost:3001/api/health</code>	Test API dari dalam VPS
<code>journalctl -u docker -f</code>	Lihat log service Docker system

11.2 Cek & Kelola Database via Terminal

■ Masuk ke PostgreSQL container

```
docker exec -it myapp-db-1 psql -U user -d myapp
```

-- Di dalam psql:

```
\dt          -- lihat semua tabel
\d users     -- struktur tabel users
SELECT COUNT(*) FROM users; -- hitung total user
SELECT * FROM users LIMIT 5;
\q          -- keluar
```

■ Restore backup database

```
gunzip -c backups/db_20241201.sql.gz \
| docker exec -i myapp-db-1 psql -U user -d myapp
```

11.3 Update Aplikasi — Zero-downtime Workflow

■ Flow update fitur baru

```
git checkout develop
git checkout -b feature/nama-fitur

# ... koding dan test lokal ...

git add .
git commit -m 'feat: nama fitur'
git push origin feature/nama-fitur

# Buat Pull Request ke develop di GitHub
# Merge develop → staging → test di staging
# Merge staging → main → GitHub Actions deploy otomatis
```

11.4 Update Dependencies & Security Patch

■ Lakukan setiap bulan

```
# Cek vulnerabilities
npm audit

# Fix otomatis (hati-hati breaking change)
npm audit fix

# Cek update tersedia
npx npm-check-updates

# Update minor/patch saja (lebih aman)
npx npm-check-updates -u --target minor
npm install

# Selalu test dulu sebelum commit!
npm run build && docker compose up -d
```

11.5 Database Optimization

■ PostgreSQL — query lambat dan index

```
-- Di dalam psql:
-- Aktifkan log query lambat (> 1 detik)
ALTER SYSTEM SET log_min_duration_statement = 1000;
SELECT pg_reload_conf();

-- Bersihkan dead rows dan update statistik
VACUUM ANALYZE;

-- Cek ukuran tabel
SELECT relname, pg_size_pretty(pg_total_relation_size(relid))
FROM pg_stat_user_tables ORDER BY pg_total_relation_size(relid) DESC;
```

■ Prisma — tambah index di schema.prisma

```
model User {
  id      String @id @default(cuid())
  email   String @unique           // sudah ada index
  role    Role

  @@index([role])                  // tambah index filter by role
}

npx prisma migrate dev --name add_role_index
```


11.6 Scaling Strategy

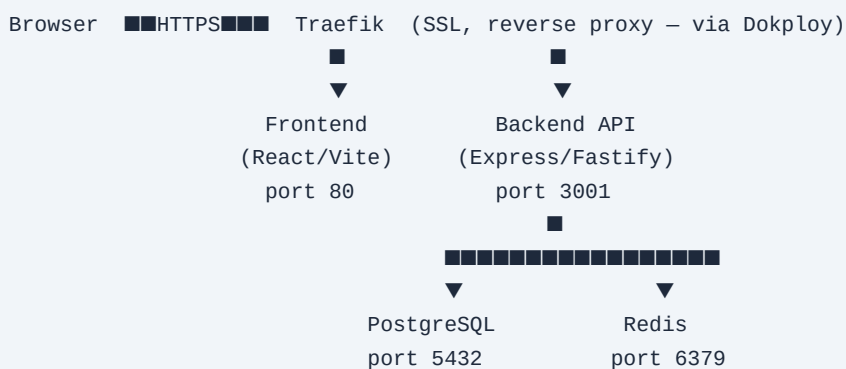
Tanda Butuh Scale	Solusi
CPU > 80% terus-menerus	Upgrade VPS atau tambah worker/replica
RAM > 90%	Tambah swap atau upgrade RAM
Response time > 1 detik	Tambah Redis caching, optimasi query DB
Disk > 80%	Bersihkan log & image lama, tambah storage
1000+ concurrent user	Load balancer, CDN untuk aset statis
Database lambat	Index, connection pool (pgBouncer), read replica

11.7 Checklist Maintenance Rutin

Frekuensi	Task Wajib
Harian	Cek docker logs error, cek disk (df -h)
Mingguan	npm audit, review Dokploy dashboard, tes restore backup
Bulanan	Update dependencies, review query lambat, prune Docker images
Per release	Tag versi git, update CHANGELOG.md, test staging dulu

Arsitektur Production Final

■ Stack yang direkomendasikan



Semua container dikelola: Docker Compose → Dokploy → VPS Ubuntu

```
CI/CD:  Push main → GitHub Actions
        → Build Docker image
        → Push ke GHCR
        → Trigger Dokploy webhook
        → Rolling deploy otomatis
```

Ingat: Pisahkan frontend dan backend container sejak hari pertama. Jangan taruh database dalam container yang sama dengan aplikasi. Pakai Docker Compose dari awal development, bukan hanya saat production. Jangan develop monolith kalau target jangka panjangnya adalah agency atau SaaS.